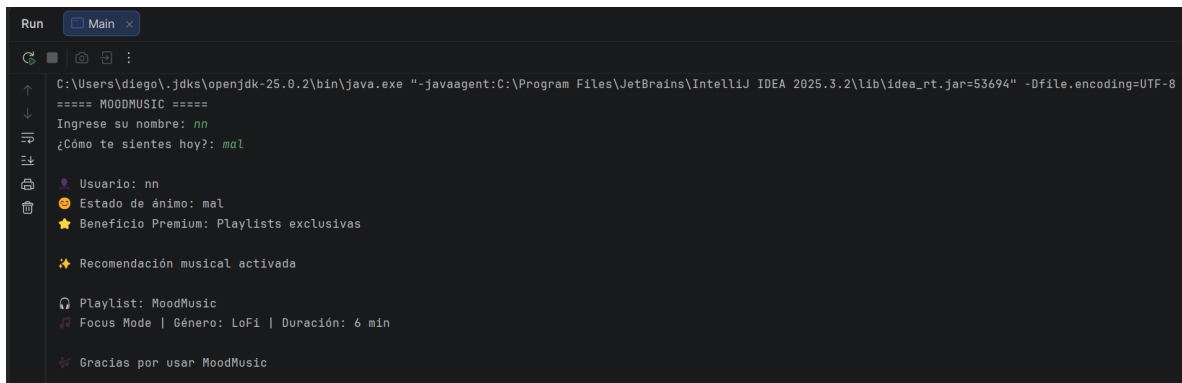


## 1. Introducción

MoodMusic se trata de un proyecto java enfocado en los pilares de Programación Orientada a Objetos. Su meta primordial es sugerir listas de reproducción musicales acorde a la emoción que el usuario proporcionará.

Este sistema opera a través de consola permitiendo así exhibir principios esenciales de POO como lo son encapsulamiento, herencia, polimorfismo, interfaz, clases, objetos y la correcta distribución de funcionalidad por paquetes.

Adicionalmente, este trabajo fue concebido con una arquitectura prolija y ordeada para facilitar su entera comprensión al momento de socializar.



```
Run Main x
C:\Users\diego\.jdk\openjdk-25.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.2\lib\idea_rt.jar=53694" -Dfile.encoding=UTF-8
==== MOODMUSIC ====
Ingrese su nombre: nn
¿Cómo te sientes hoy?: mal

Usuario: nn
Estado de ánimo: mal
Beneficio Premium: Playlists exclusivas

Recomendación musical activada

Playlist: MoodMusic
Focus Mode | Género: LoFi | Duración: 6 min

Gracias por usar MoodMusic
```

## 2. Objetivo General

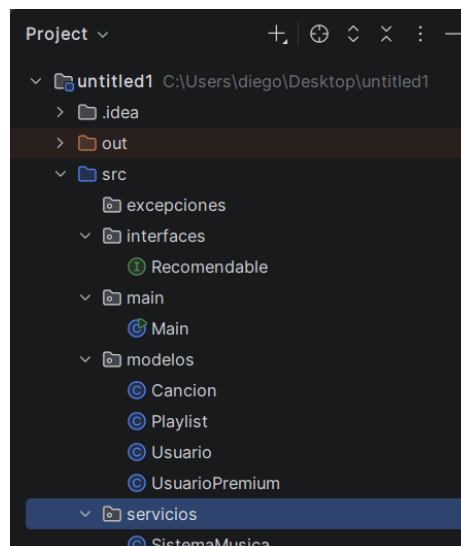
Desarrollar un sistema funcional en Java que recomiende playlists musicales dependiendo del estado emocional del usuario, aplicando los fundamentos de Programación Orientada a Objetos.

## 3. Estructura General del Proyecto

El proyecto se encuentra dividido en paquetes para mantener una organización adecuada del código.

src

- |
- |— excepciones
- |— interfaces
- |— main
- |— modelos
- |— servicios



## 4. Explicación de los Paquetes

### 4.1 Paquete interfaces

Este paquete contiene las interfaces del sistema. Las interfaces permiten definir comportamientos que otras clases deben implementar.

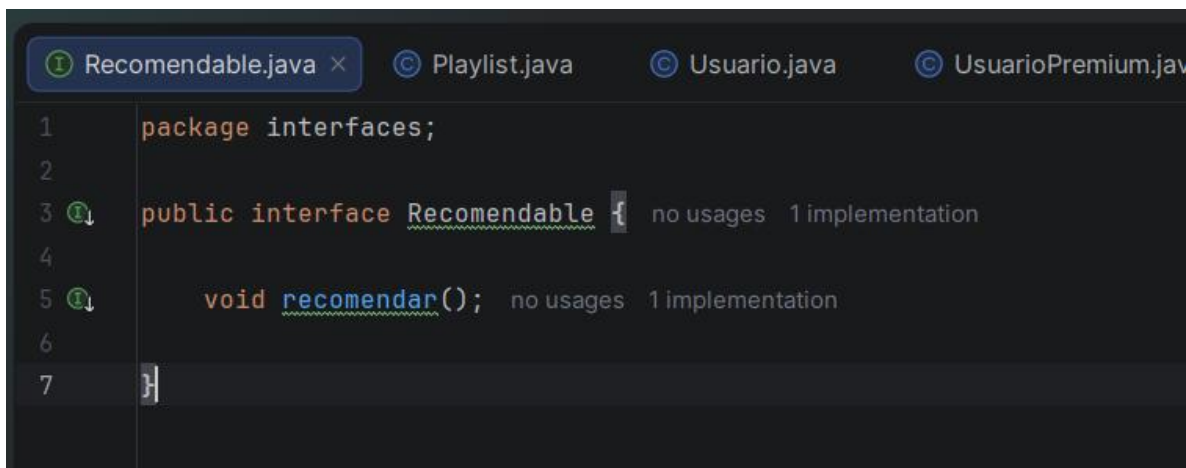
En este proyecto se creó la interfaz:

Recomendable.java

La función de esta interfaz es obligar a que las clases que la implementen tengan el método:

`void recomendar();`

Esto ayuda a aplicar abstracción y polimorfismo.



```
1 package interfaces;
2
3 public interface Recomendable { no usages 1 implementation
4
5     void recomendar(); no usages 1 implementation
6
7 }
```

### 4.2 Paquete modelos

Este paquete contiene las clases principales del sistema, es decir, los modelos que representan los objetos reales del programa.

Clases incluidas:

- Cancion.java
- Playlist.java
- Usuario.java
- UsuarioPremium.java

Estas clases almacenan atributos, constructores y métodos relacionados con el funcionamiento del sistema.

### 4.3 Paquete servicios

Este paquete contiene la lógica principal del sistema.

Clase incluida:

SistemaMusica.java

Esta clase se encarga de generar playlists según el estado de ánimo ingresado por el usuario.

La separación de esta lógica permite que el proyecto sea más organizado y fácil de mantener.

#### **4.4 Paquete main**

Este paquete contiene la clase principal del programa:

Main.java

Desde esta clase inicia toda la ejecución del sistema.

Sus funciones principales son:

- solicitar datos al usuario
- crear objetos
- ejecutar métodos
- mostrar resultados en consola

#### **4.5 Paquete excepciones**

Este paquete fue creado para almacenar excepciones personalizadas del sistema.

Aunque actualmente el proyecto utiliza validaciones básicas, este paquete permite ampliar el manejo de errores en futuras mejoras.

### **5. Explicación de Cada Clase**

#### **5.1 Clase Recomendable.java**

Esta interfaz define el método:

recomendar()

Cualquier clase que implemente esta interfaz estará obligada a utilizar dicho método.

Esto permite aplicar:

- abstracción
- polimorfismo

#### **5.2 Clase Cancion.java**

La clase Cancion representa una canción dentro del sistema.

#### **Atributos**

titulo  
genero  
duracion

#### **Función**

Permite almacenar y mostrar información de cada canción.

#### **Métodos principales**

mostrarCancion()

Este método imprime los datos de la canción en consola.

### **5.3 Clase Playlist.java**

La clase Playlist representa una lista de canciones.

#### **Funciones principales**

- almacenar canciones
- mostrar playlists
- recomendar música

#### **Colección utilizada**

ArrayList<Cancion>

Esto permite almacenar múltiples canciones dinámicamente.

#### **Métodos importantes**

agregarCancion()  
mostrarPlaylist()  
recomendar()

### **5.4 Clase Usuario.java**

Representa a los usuarios del sistema.

#### **Atributos**

nombre  
estadoAnimo

#### **Función**

Guardar la información básica del usuario.

### **Método principal**

mostrarUsuario()

### **5.5 Clase UsuarioPremium.java**

Esta clase hereda de Usuario.

#### **Función**

Agregar características premium al usuario.

#### **Atributo adicional**

beneficio

#### **Método adicional**

mostrarBeneficio()

### **5.6 Clase SistemaMusica.java**

Esta clase contiene la lógica principal del programa.

#### **Función**

Generar playlists según el estado de ánimo ingresado.

#### **Método principal**

generarPlaylist()

Dependiendo del estado:

- feliz
- triste
- motivado

el sistema genera canciones diferentes.

### **5.7 Clase Main.java**

Es la clase principal del sistema.

#### **Funciones**

- iniciar el programa
- pedir datos al usuario
- crear objetos
- ejecutar métodos

- mostrar resultados

### Objetos creados

UsuarioPremium

SistemaMusica

Playlist

## 6. Aplicación de los Fundamentos de POO

### 6.1 Clases y Objetos

El proyecto utiliza múltiples clases para representar elementos reales del sistema.

Ejemplos:

- Usuario
- Cancion
- Playlist

También se crean objetos a partir de estas clases.

Ejemplo:

```
Playlist playlist = new Playlist("MoodMusic");
```

### 6.2 Encapsulamiento

Los atributos de las clases fueron declarados privados o protegidos para proteger la información.

Ejemplo:

```
private String titulo;
```

El acceso se realiza mediante métodos públicos.

```
package modelos;

public class Cancion { no usages

    private String titulo; 3 usages
    private String genero; 3 usages
    private int duracion; 3 usages

    public Cancion(String titulo, String genero, int duracion) { no usages
```

### 6.3 Constructores

Todas las clases utilizan constructores para inicializar objetos.

Ejemplo:

```
public Cancion(String titulo, String genero, int duracion)
```

### 6.4 Herencia

La herencia se aplica en:

UsuarioPremium extends Usuario

Esto permite reutilizar atributos y métodos de la clase padre.

### 6.5 Polimorfismo

El polimorfismo se evidencia mediante la implementación de interfaces y el uso de métodos sobrescritos.

Ejemplo:

```
@Override  
public void recomendar()
```

### 6.6 Abstracción

La abstracción se aplica mediante la interfaz:

Recomendable

La interfaz define comportamientos generales sin mostrar detalles internos.

### 6.7 Organización Modular

El proyecto se encuentra dividido en paquetes para separar responsabilidades y mejorar la organización del código.

Esto facilita:

- mantenimiento
- lectura
- comprensión
- escalabilidad